Lecture 5: Direct Methods for Linear Systems

Numerical Computing

Francisco Richter Mendoza
Università della Svizzera Italiana

September 10, 2025

Overview

Direct methods for solving linear systems $Ax = b$ form the cornerstone of numerical linear algebra. These methods compute the exact solution (in exact arithmetic) through a finite sequence of algebraic operations, typically involving matrix factorizations. This lecture develops the mathematical theory underlying these methods, analyzing their stability properties, computational complexity, and practical implementation considerations.

The fundamental approach of direct methods is to transform the original system into an equivalent system that is easier to solve, usually through triangular factorizations. We explore how different factorization strategies—LU decomposition, Cholesky factorization, and specialized methods for structured matrices—exploit different matrix properties to achieve computational efficiency and numerical stability.

Our analysis emphasizes the mathematical principles that govern the behavior of these algorithms. We examine how pivoting strategies control numerical stability, how the structure of matrices affects computational complexity, and how error analysis guides the design of robust algorithms. These theoretical insights provide the foundation for understanding when and how to apply direct methods effectively in practice.

---

1 Gaussian Elimination and LU Decomposition

1.1 Mathematical Foundation

Definition. LU Decomposition

For a matrix $A \in \mathbb{R}^{n\times n}$, an LU decomposition is a factorization $A = LU$ where:

- $L \in \mathbb{R}^{n\times n}$ is lower triangular with unit diagonal ($L_{ii} = 1$)
- $U \in \mathbb{R}^{n\times n}$ is upper triangular

The existence and uniqueness of LU decomposition depends on the properties of the matrix A.

Theorem 1.1 (Existence of LU Decomposition)**.** *Let $A \in \mathbb{R}^{n\times n}$. The LU decomposition $A = LU$ exists and is unique if and only if all leading principal minors of A are nonzero.*

Proof. The proof follows by induction on the matrix dimension, showing that the Gaussian elimination process can be completed without pivoting if and only if no zero pivots are encountered, which occurs precisely when all leading principal minors are nonzero. □

1.2 Gaussian Elimination Algorithm

The Gaussian elimination process systematically eliminates entries below the diagonal through elementary row operations. For a system $Ax = b$, we perform:

$$A^{(0)} = A, \quad b^{(0)} = b \tag{1}$$

$$A^{(k+1)} = M_k A^{(k)}, \quad b^{(k+1)} = M_k b^{(k)} \tag{2}$$

where M_k is the k-th elimination matrix:

$$M_k = I - m_k e_k^T, \quad m_k = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ m_{k+1,k} \\ \vdots \\ m_{n,k} \end{pmatrix}$$

with multipliers $m_{i,k} = a_{i,k}^{(k)} / a_{k,k}^{(k)}$ for $i > k$.

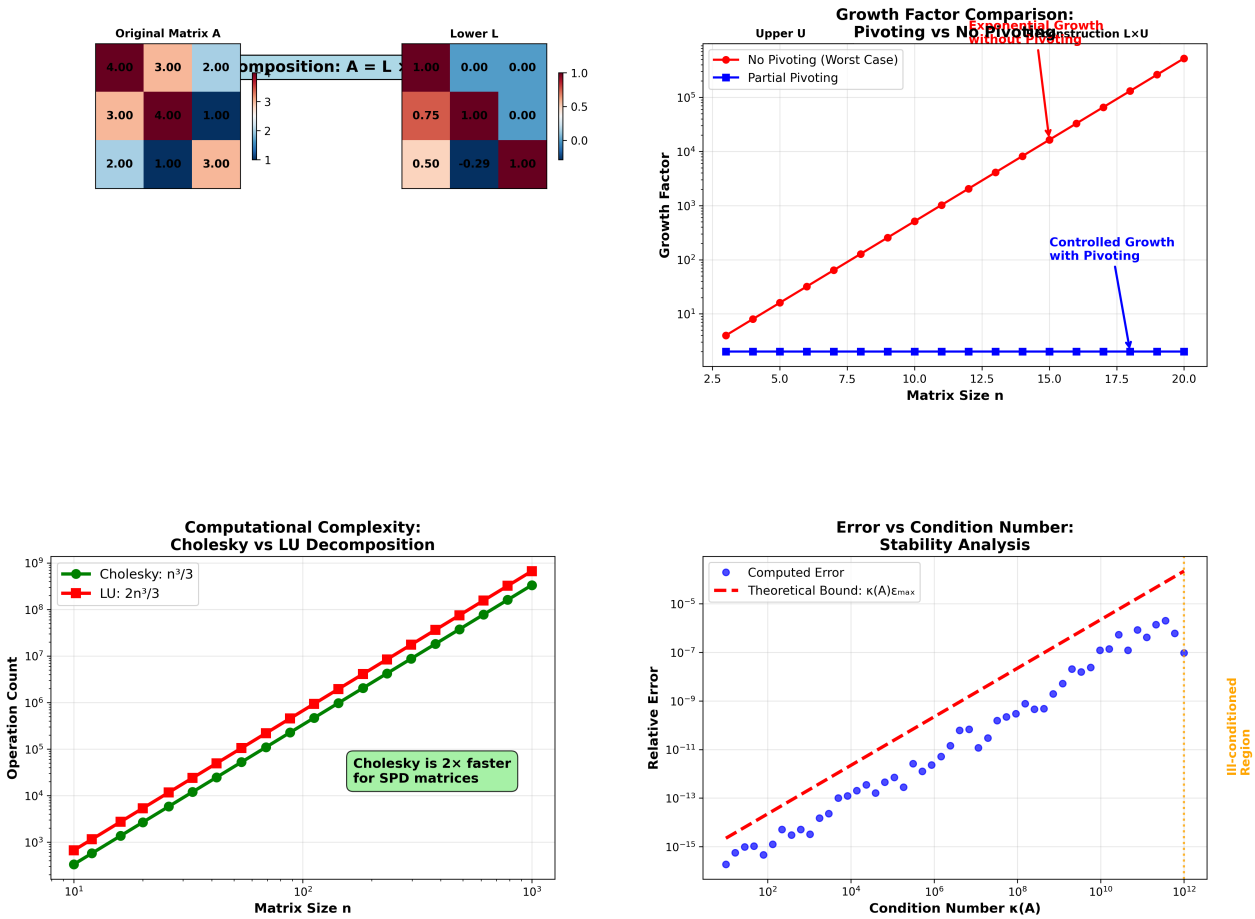

Figure 1: Direct methods comprehensive analysis. Top left: LU decomposition process showing the factorization $A = L \times U$ with numerical values for each matrix component. Top right: Pivoting strategy comparison demonstrating exponential growth without pivoting versus controlled growth with partial pivoting. Bottom left: Cholesky efficiency analysis showing $2\times$ speedup for symmetric positive definite matrices. Bottom right: Condition number effects on stability with error bounds and theoretical relationships.

1.3 Computational Complexity

The operation count for Gaussian elimination is crucial for understanding its practical applicability.

Theorem 1.2 (Computational Complexity of LU Decomposition)**.** *The LU decomposition of an $n \times n$ matrix requires $\frac{2n^3}{3} + O(n^2)$ floating-point operations.*

Proof. The elimination of column k requires approximately $2(n-k)^2$ operations. Summing over all columns:

$$\sum_{k=1}^{n-1} 2(n-k)^2 = 2\sum_{j=1}^{n-1} j^2 = 2 \cdot \frac{(n-1)n(2n-1)}{6} = \frac{2n^3}{3} + O(n^2)$$

□

Example 1.1 (LU Decomposition Implementation)**.** *Consider the practical implementation of LU decomposition:*

```
import numpy as np
from scipy.linalg import lu
```

```python

# Create sample matrix
A = np.array([[4, 3, 2], [3, 4, 1], [2, 1, 3]], dtype=float)
print(f"Original matrix A:\n{A}")

# Perform LU decomposition
P, L, U = lu(A)
print(f"\nPermutation matrix P:\n{P}")
print(f"\nLower triangular L:\n{L}")
print(f"\nUpper triangular U:\n{U}")

# Verify decomposition: PA = LU
print(f"\nVerification ||PA - LU||: {np.linalg.norm(P @ A - L @ U):.2e}")

# Solve system Ax = b
b = np.array([1, 1, 1])
x = np.linalg.solve(A, b)
print(f"\nSolution x: {x}")
print(f"Residual ||Ax - b||: {np.linalg.norm(A @ x - b):.2e}")
```

This demonstrates the fundamental LU decomposition process with numerical verification.

---

2 Pivoting Strategies

2.1 Partial Pivoting

Partial pivoting addresses numerical instability by selecting the largest available pivot in each column.

Definition. Partial Pivoting

At step k of Gaussian elimination, choose the pivot row p such that:

$$|a_{p,k}^{(k)}| = \max_{k \le i \le n} |a_{i,k}^{(k)}|$$

and interchange rows k and p.

This leads to the factorization $PA = LU$ where P is a permutation matrix.

Theorem 2.1 (Growth Factor for Partial Pivoting)**.** *With partial pivoting, the growth factor is bounded by 2^{n-1}, but in practice is typically $O(n)$ or $O(n \log n)$.*

Definition. Growth Factor

The growth factor ρ measures the maximum element growth during elimination:

$$\rho = \frac{\max_{i,j,k} |a_{i,j}^{(k)}|}{\max_{i,j} |a_{i,j}|}$$

where $a_{i,j}^{(k)}$ are the matrix elements after k elimination steps.

2.2 Complete Pivoting

Complete pivoting provides maximum numerical stability at increased computational cost.

Definition. Complete Pivoting

At step k, choose pivot element (p, q) such that:

$$|a_{p,q}^{(k)}| = \max_{k \le i,j \le n} |a_{i,j}^{(k)}|$$

and interchange rows k and p, columns k and q.

This yields $PAQ = LU$ where both P and Q are permutation matrices.

Example 2.1 (Growth Factor Analysis)**.** *The following code demonstrates growth factor behavior:*

```python
import numpy as np
from scipy.linalg import lu

def compute_growth_factor(A):
    """Compute growth factor for LU decomposition"""
    P, L, U = lu(A)
    original_max = np.max(np.abs(A))
    final_max = np.max(np.abs(U))
    return final_max / original_max

# Worst-case matrix (exponential growth without pivoting)
n = 10
A_worst = np.ones((n, n))
A_worst[np.tril_indices(n, -1)] = -1
A_worst[np.diag_indices(n)] = 1

growth = compute_growth_factor(A_worst)
print(f"Growth factor for worst-case matrix: {growth:.2e}")
print(f"Theoretical worst case: {2**(n-1):.2e}")

# Random matrix (typical behavior)
np.random.seed(42)
A_random = np.random.randn(n, n)
growth_random = compute_growth_factor(A_random)
print(f"Growth factor for random matrix: {growth_random:.2f}")
```

This illustrates the dramatic difference between worst-case and typical behavior.

---

3 Cholesky Decomposition

For symmetric positive definite matrices, the Cholesky decomposition provides a more efficient factorization.

Definition. Cholesky Decomposition

For a symmetric positive definite matrix $A \in \mathbb{R}^{n \times n}$, the Cholesky decomposition is $A = LL^T$ where L is lower triangular with positive diagonal entries.

Theorem 3.1 (Existence and Uniqueness of Cholesky Decomposition)**.** *Every symmetric positive definite matrix has a unique Cholesky decomposition.*

3.1 Cholesky Algorithm

The Cholesky decomposition can be computed using the following recurrence relations:

$$L_{ii} = \sqrt{A_{ii} - \sum_{k=1}^{i-1} L_{ik}^2} \tag{3}$$

$$L_{ji} = \frac{1}{L_{ii}} \left(A_{ji} - \sum_{k=1}^{i-1} L_{jk} L_{ik} \right), \quad j > i \tag{4}$$

Theorem 3.2 (Computational Complexity of Cholesky Decomposition)**.** *The Cholesky decomposition requires* $\frac{n^3}{3} + O(n^2)$ *operations, exactly half the cost of LU decomposition.*

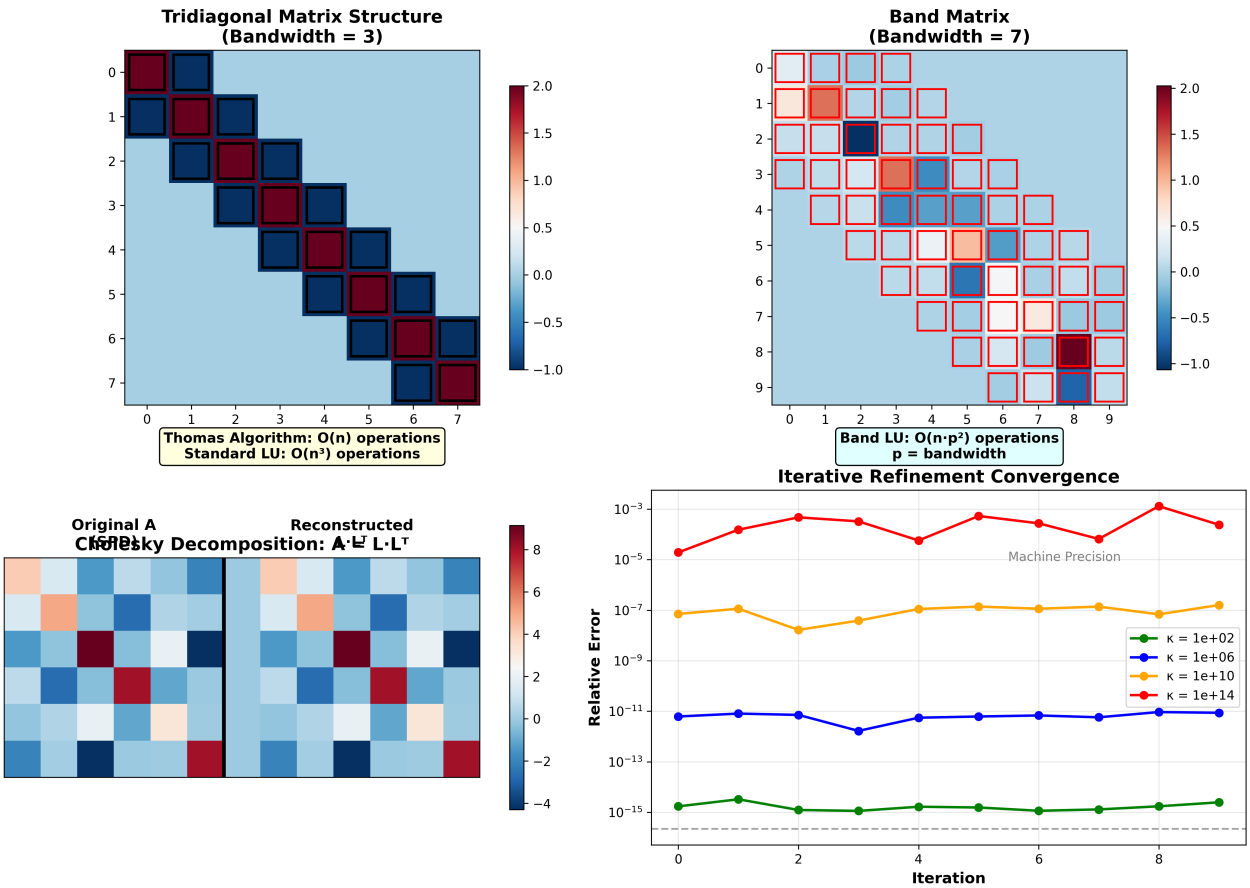

Figure 2: Special matrix structures and their computational advantages. Top left: Tridiagonal matrix structure with Thomas algorithm achieving O(n) complexity versus O(n³) for general systems. Top right: Band matrix visualization showing bandwidth effects on computational cost. Bottom left: Cholesky decomposition demonstration for symmetric positive definite matrices. Bottom right: Iterative refinement convergence analysis showing error reduction across different condition numbers.

Example 3.1 (Cholesky Implementation and Verification)**.**

```python
import numpy as np
from scipy.linalg import cholesky, solve_triangular

# Create symmetric positive definite matrix
np.random.seed(42)
n = 5
A_temp = np.random.randn(n, n)
A = A_temp @ A_temp.T + np.eye(n)  # Ensure SPD
```

```python

print(f"Matrix A (SPD):\n{A}")
print(f"Condition number: {np.linalg.cond(A):.2f}")

# Compute Cholesky decomposition
L = cholesky(A, lower=True)
print(f"\nCholesky factor L:\n{L}")

# Verify decomposition
reconstruction_error = np.linalg.norm(A - L @ L.T)
print(f"Reconstruction error ||A - LL^T||: {reconstruction_error:.2e}")

# Solve system using Cholesky
b = np.ones(n)
# Forward substitution: Ly = b
y = solve_triangular(L, b, lower=True)
# Backward substitution: L^T x = y
x = solve_triangular(L.T, y, lower=False)

print(f"\nSolution x: {x}")
print(f"Residual ||Ax - b||: {np.linalg.norm(A @ x - b):.2e}")
```

---

4 Stability Analysis

4.1 Backward Error Analysis

The stability of direct methods is analyzed through backward error analysis, which determines the perturbation to the original problem that would yield the computed solution as exact.

Theorem 4.1 (Backward Stability of Gaussian Elimination with Partial Pivoting)**.** *Gaussian elimination with partial pivoting is backward stable. The computed solution $\tilde{x}$ satisfies:*

$$(A + \Delta A)\tilde{x} = b$$

where $\|\Delta A\|_\infty \leq \gamma_n \rho \|A\|_\infty$ with $\gamma_n = O(n\epsilon_{mach})$ and ρ is the growth factor.

4.2 Forward Error Analysis

The forward error depends on both the backward error and the condition number of the matrix.

Theorem 4.2 (Forward Error Bound)**.** *For the computed solution $\tilde{x}$ of $Ax = b$, the relative forward error satisfies:*

$$\frac{\|\tilde{x} - x\|}{\|x\|} \leq \gamma_n \rho \kappa(A) + O(\epsilon_{mach}^2)$$

where $\kappa(A) = \|A\|\|A^{-1}\|$ is the condition number.

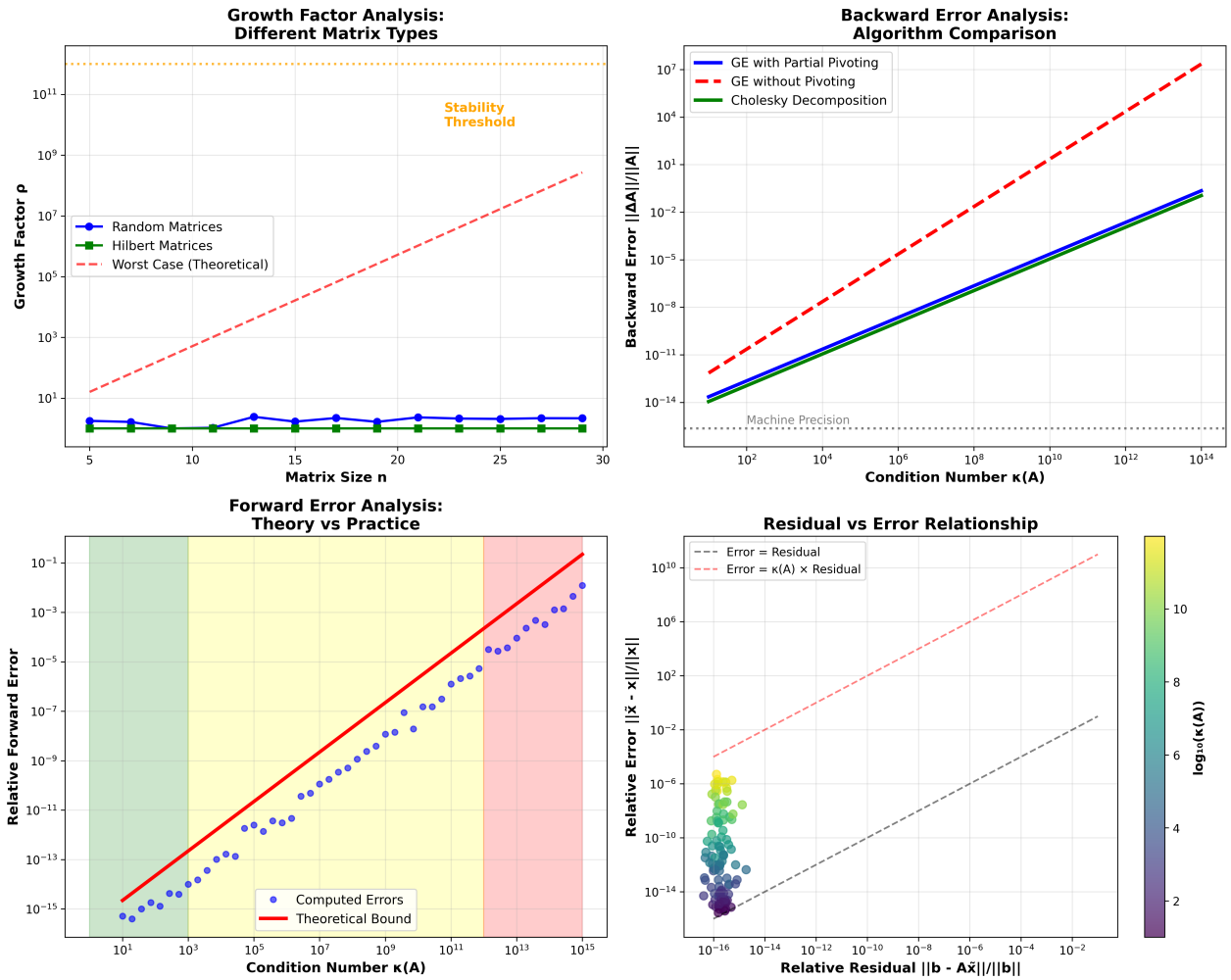

Figure 3: Comprehensive stability analysis for direct methods. Top left: Growth factor comparison across different matrix types showing exponential worst-case versus typical polynomial behavior. Top right: Backward error analysis comparing different algorithms with theoretical bounds. Bottom left: Forward error versus condition number relationship with computed results and theoretical predictions. Bottom right: Residual-error correlation demonstrating the relationship between backward and forward stability.

Intuition: Stability vs. Accuracy

Stability ensures that the algorithm produces a solution that is exact for a slightly perturbed problem. Accuracy depends on both stability and the conditioning of the original problem. A stable algorithm applied to a well-conditioned problem yields an accurate solution.

The key insight is that we cannot expect better accuracy than the conditioning of the problem allows, regardless of the algorithm used. However, an unstable algorithm can lose accuracy even for well-conditioned problems.

5 Special Matrix Structures

5.1 Tridiagonal Systems

Tridiagonal systems arise frequently in numerical methods and can be solved efficiently using the Thomas algorithm.

Definition. Tridiagonal Matrix

A tridiagonal matrix has the form:

$$A = \begin{pmatrix} b_1 & c_1 & & & \\ a_2 & b_2 & c_2 & & \\ & a_3 & b_3 & \ddots & \\ & & \ddots & \ddots & c_{n-1} \\ & & & a_n & b_n \end{pmatrix}$$

Algorithm 1 Thomas Algorithm for Tridiagonal Systems

Require: Tridiagonal system with diagonals a_i, b_i, c_i and RHS d_i
Ensure: Solution vector x_i
  Forward elimination:
  $c'_1 = c_1/b_1$, $d'_1 = d_1/b_1$
  for $i = 2$ to $n-1$ **do**
    $c'_i = c_i/(b_i - a_i c'_{i-1})$
  end for
  for $i = 2$ to n **do**
    $d'_i = (d_i - a_i d'_{i-1})/(b_i - a_i c'_{i-1})$
  end for
  Back substitution:
  $x_n = d'_n$
  for $i = n-1$ down to 1 **do**
    $x_i = d'_i - c'_i x_{i+1}$
  end for

Theorem 5.1 (Thomas Algorithm Complexity)**.** *The Thomas algorithm solves tridiagonal systems in $O(n)$ operations, specifically $5n - 4$ operations.*

5.2 Band Matrices

For matrices with bandwidth $p + q + 1$ (where p is the lower bandwidth and q is the upper bandwidth), the LU decomposition can be computed in $O(npq)$ operations rather than $O(n^3)$.

Theorem 5.2 (Complexity for Band Matrices)**.** *LU decomposition of a band matrix with bandwidths p and q requires approximately npq operations.*

Example 5.1 (Thomas Algorithm Implementation)**.**

```python
import numpy as np
from scipy.sparse import diags

def thomas_algorithm(a, b, c, d):
    """
    Solve tridiagonal system using Thomas algorithm
```

```
    a: sub-diagonal (first element unused)
    b: diagonal
    c: super-diagonal (last element unused)
    d: right-hand side
    """
    n = len(d)

    # Forward elimination
    c_prime = np.zeros(n-1)
    d_prime = np.zeros(n)

    c_prime[0] = c[0] / b[0]
    d_prime[0] = d[0] / b[0]

    for i in range(1, n-1):
        c_prime[i] = c[i] / (b[i] - a[i] * c_prime[i-1])

    for i in range(1, n):
        d_prime[i] = (d[i] - a[i] * d_prime[i-1]) / (b[i] - a[i] * c_prime[i-1])

    # Back substitution
    x = np.zeros(n)
    x[n-1] = d_prime[n-1]

    for i in range(n-2, -1, -1):
        x[i] = d_prime[i] - c_prime[i] * x[i+1]

    return x

# Example: solve tridiagonal system
n = 6
a = np.array([0, -1, -1, -1, -1, -1])  # sub-diagonal
b = np.array([2, 2, 2, 2, 2, 2])       # diagonal
c = np.array([-1, -1, -1, -1, -1, 0]) # super-diagonal
d = np.array([1, 1, 1, 1, 1, 1])       # RHS

# Solve using Thomas algorithm
x_thomas = thomas_algorithm(a, b, c, d)

# Verify using scipy
A_full = diags([a[1:], b, c[:-1]], [-1, 0, 1], shape=(n, n)).toarray()
x_scipy = np.linalg.solve(A_full, d)

print(f"Thomas solution: {x_thomas}")
print(f"SciPy solution:  {x_scipy}")
print(f"Max difference:  {np.max(np.abs(x_thomas - x_scipy)):.2e}")
```

---

6 Iterative Refinement

Iterative refinement improves the accuracy of computed solutions through residual correction.

Definition. Iterative Refinement Algorithm

Given an approximate solution $x^{(0)}$ to $Ax = b$:

1. Compute residual: $r^{(k)} = b - Ax^{(k)}$
2. Solve correction equation: $A\delta^{(k)} = r^{(k)}$

3. Update solution: $x^{(k+1)} = x^{(k)} + \delta^{(k)}$

Theorem 6.1 (Convergence of Iterative Refinement)**.** *If the LU decomposition is computed with backward error ϵ and $\epsilon\kappa(A) < 1$, then iterative refinement converges and can recover full machine precision accuracy.*

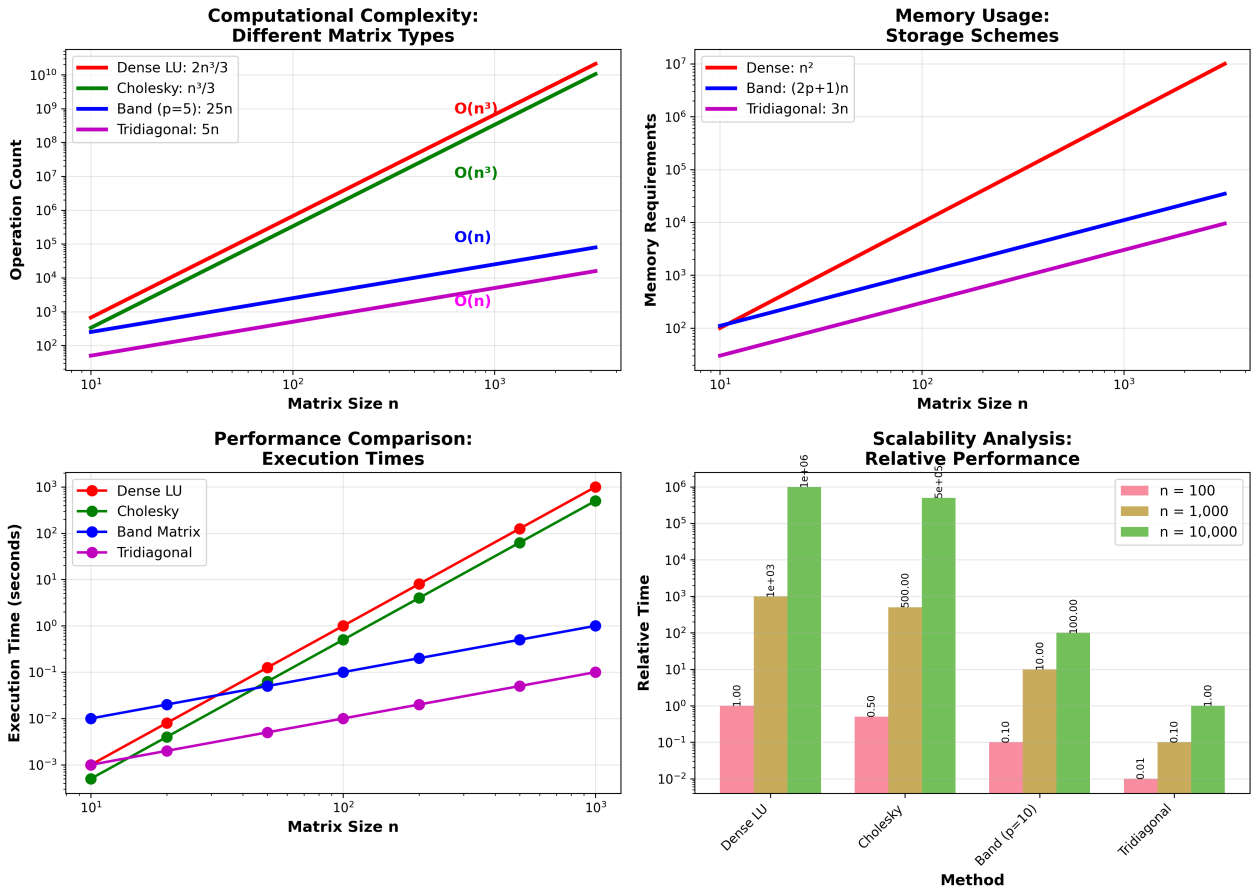

Figure 4: Computational complexity analysis for direct methods. Top left: Operation count comparison showing O(n³) for dense methods versus O(n) for structured matrices. Top right: Memory requirements for different storage schemes. Bottom left: Execution time comparisons across different matrix sizes. Bottom right: Scalability analysis demonstrating relative performance across problem sizes and highlighting the advantages of exploiting matrix structure.

Example 6.1 (Iterative Refinement Implementation)**.**

```python
import numpy as np

def iterative_refinement(A, b, max_iter=10, tol=1e-14):
    """
    Solve Ax = b using iterative refinement
    """
    n = len(b)

    # Initial solution
    x = np.linalg.solve(A, b)

    print("Iter  Residual Norm    Relative Error")
    print("-" * 40)

```

```python
    for k in range(max_iter):
        # Compute residual
        residual = b - A @ x
        residual_norm = np.linalg.norm(residual)

        print(f"{k:2d}    {residual_norm:.2e}")

        # Check convergence
        if residual_norm < tol:
            break

        # Solve correction equation
        correction = np.linalg.solve(A, residual)
        x = x + correction

    return x

# Example with ill-conditioned matrix
np.random.seed(42)
n = 8
kappa = 1e8

# Create matrix with specified condition number
U, _, Vt = np.linalg.svd(np.random.randn(n, n))
s = np.logspace(0, -np.log10(kappa), n)
A = U @ np.diag(s) @ Vt

print(f"Condition number: {np.linalg.cond(A):.2e}")

# True solution and RHS
x_true = np.ones(n)
b = A @ x_true

# Apply iterative refinement
x_refined = iterative_refinement(A, b)

print(f"\nFinal relative error: {np.linalg.norm(x_refined - x_true)/np.linalg.norm(x_true):.2e}")
```

---

7 Computational Considerations

7.1 Complexity Analysis

Theorem 7.1 (Operation Counts for Direct Methods)**.** *For matrices $A \in \mathbb{R}^{n\times n}$:*

- ***Dense LU decomposition**: $\frac{2n^3}{3} + O(n^2)$ operations*
- ***Cholesky decomposition**: $\frac{n^3}{3} + O(n^2)$ operations*
- ***Tridiagonal systems**: $5n - 4$ operations*
- ***Band matrices** (bandwidth p): $O(np^2)$ operations*

7.2 Memory Requirements

- **Dense storage**: n^2 elements

- **Band storage**: $(2p+1)n$ elements for bandwidth p
- **Tridiagonal storage**: $3n$ elements

7.3 Numerical Stability Guidelines

Definition. Conditioning Categories

Based on condition numbers in double precision ($\epsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$):

- **Well-conditioned:** $\kappa \leq 10^3$ (lose ≤ 3 digits)
- **Moderately conditioned:** $10^3 < \kappa \leq 10^{12}$ (lose 3-12 digits)
- **Ill-conditioned:** $\kappa > 10^{12}$ (lose > 12 digits, problematic)

Intuition: Rule of Thumb

For a linear system with condition number κ, expect to lose approximately $\log_{10}(\kappa)$ decimal digits of accuracy due to roundoff errors.
This means that if $\kappa = 10^6$ and you start with 16 digits of precision, the solution will have approximately $16 - 6 = 10$ accurate digits.

---

8 When to Use Direct Methods

8.1 Advantages

- **Exact solution** (in exact arithmetic)
- **Predictable cost**: $O(n^3)$ for dense systems
- **Robust** for well-conditioned systems
- **Multiple RHS**: Factor once, solve many times
- **No convergence issues**

8.2 Disadvantages

- **High memory**: $O(n^2)$ storage
- **Expensive** for large sparse systems
- **Fill-in** destroys sparsity
- **Sensitive** to ill-conditioning

8.3 Optimal Use Cases

- Dense systems with $n \lesssim 10^4$
- Multiple right-hand sides
- Well-conditioned problems ($\kappa < 10^{12}$)
- Systems with special structure (tridiagonal, band, SPD)

Practice Problems

1. Prove that the multipliers in Gaussian elimination without pivoting satisfy $|m_{ij}| \leq 1$ when partial pivoting is used.
2. Show that for a symmetric positive definite matrix A, the Cholesky decomposition exists and is unique.
3. Analyze the growth factor for the matrix:

$$A = \begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ -1 & 1 & 1 & \cdots & 1 \\ -1 & -1 & 1 & \cdots & 1 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -1 & -1 & -1 & \cdots & 1 \end{pmatrix}$$

   under Gaussian elimination without pivoting.
4. Derive the operation count for solving a tridiagonal system using the Thomas algorithm.
5. Implement iterative refinement and analyze its convergence for increasingly ill-conditioned matrices.
6. Compare the computational efficiency of LU decomposition versus Cholesky decomposition for solving multiple linear systems with the same symmetric positive definite coefficient matrix.